Distributed Mutual Exclusion Explorer

Interactive Tools for Learning Distributed Systems Concepts (Variant 3)

Author: Lei Ding

Student ID: K21029011

Programme: BSc Computer Science

Supervisor: Angelos Georgoulas

Department of Informatics, King's College London

March 14, 2026

Originality Avowal

This report is submitted as part of the requirements for the Final Year Individual Project. It is the result of the author's own work except where explicitly indicated in the text. All sources are acknowledged by appropriate references. This report has not been submitted for any other assessment at this or any other institution.

Abstract

Distributed mutual exclusion is a foundational distributed systems problem, yet its behaviour is difficult to learn from static descriptions because correctness depends on concurrency, ordering, and failure assumptions. This project delivers a browser-based interactive teaching tool that visualises distributed mutual exclusion using two representative approaches: a token-based Token Ring model and a teaching-oriented Ricart–Agrawala (RA) prototype. The tool supports step-by-step execution, scripted demonstrations, fault injection (token loss, crash/recover, and message loss in RA), and evidence export (state JSON, trace TXT, preview PNG) for reproducible evaluation. Correctness is evaluated primarily via the safety property (mutual exclusion), while fault scenarios are used to illustrate the separation between safety and liveness. Testing combines a manual evidence-driven test plan with dependency-free automated smoke tests executed in continuous integration. The resulting artefact provides an accessible environment for exploring distributed coordination and for producing traceable evaluation evidence.

Acknowledgements

The author would like to thank the project supervisor for guidance and feedback throughout the project.

Contents

Originality Avowal i

Abstract ii

Acknowledgements iii

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Project Aim and Objectives . . . 1
- 1.3 Scope . . . 1
- 1.4 Contributions . . . 2
- 1.5 Report Structure . . . 2

2 Background and Related Work 3
- 2.1 Distributed Mutual Exclusion . . . 3
- 2.2 Message-Based and Token-Based Approaches . . . 3
- 2.3 Logical Time and Ordering . . . 4
- 2.4 Failures, Assumptions, and the Safety–Liveness Distinction . . . 4
- 2.5 Educational Visualisation and Interactive Learning . . . 5
- 2.6 Cognitive Load, Usability, and Tool Design . . . 5
- 2.7 Reproducibility and Research Software Practice . . . 5
- 2.8 Gap and Positioning . . . 5

3 Specification and Design 7
- 3.1 Development Methodology . . . 7
- 3.2 Requirements Approach . . . 7
- 3.3 Functional Requirements . . . 7
- 3.4 Non-functional Requirements . . . 8
- 3.5 Architecture Overview . . . 8
- 3.6 Requirements-to-Design Traceability . . . 8
- 3.7 Execution Model . . . 8
- 3.8 Key Design Trade-offs . . . 9
  - 3.8.1 Queue-Based Network Abstraction . . . 9
  - 3.8.2 No Retransmission or Failure Detector . . . 9
  - 3.8.3 Browser-Only and Dependency-Free . . . 9
- 3.9 Reproducibility by Design . . . 9
- 3.10 Design Summary . . . 9

4 Implementation 10
- 4.1 Implementation Goals . . . 10

4.2 Repository Structure and Responsibilities . . . 10
4.3 Core Data Structures . . . 10
4.3.1 Process State . . . 10
4.3.2 Token Ring State . . . 11
4.3.3 RA Network State . . . 11
4.3.4 Trace and Metrics . . . 11
4.4 Safety Checking . . . 11
4.5 Step Engine . . . 11
4.5.1 Token Ring Step Semantics . . . 11
4.5.2 RA Step Semantics . . . 12
4.6 Token Ring Implementation . . . 12
4.6.1 Token Faults and Recovery . . . 12
4.7 Ricart–Agrawala Implementation . . . 12
4.7.1 Tie-break Logic . . . 12
4.8 Fault Injection Points . . . 13
4.8.1 Token Ring . . . 13
4.8.2 Ricart–Agrawala . . . 13
4.9 Stall Explanation Logic . . . 13
4.10 Scripted Scenarios . . . 13
4.11 Export Pipeline . . . 13
4.12 Implementation Limitations . . . 13

5 Testing **15**
5.1 Testing Methodology . . . 15
5.2 Testing Objectives . . . 15
5.3 Manual Evidence-Driven Testing . . . 15
5.4 Automated Testing . . . 16
5.5 Continuous Integration . . . 16
5.6 Pass Criteria and Outcome Categories . . . 16
5.7 Test Execution Summary . . . 16
5.8 Quantitative Snapshot . . . 16
5.9 Evidence Capture Workflow . . . 18
5.10 Testing Limitations . . . 18

6 Evaluation **19**
6.1 Evaluation Methodology . . . 19
6.2 Evaluation Criteria . . . 19
6.3 Correctness and Safety . . . 19
6.3.1 Token Ring Fault Cases . . . 20
6.3.2 RA Tie-break Behaviour . . . 20
6.4 Fault Demonstrations: Safety vs Liveness . . . 21
6.4.1 RA-03: Drop Next In-Flight Message . . . 21
6.4.2 RA-04: Drop-Next-Send . . . 22
6.5 Usability Evaluation . . . 22
6.6 Reproducibility . . . 23

6.7 Threats to Validity . . . 23
6.8 Summary . . . 24

7 Professional Issues **25**
7.1 Overview . . . 25
7.2 Accessibility . . . 25
7.3 Privacy and Data Handling . . . 26
7.4 Ethics and Research Governance . . . 26
7.5 Sustainability and Maintainability . . . 26
7.6 Reproducibility and Research Integrity . . . 27
7.7 Responsible Communication of Limitations . . . 27
7.8 Professional Standards and Evaluation Practice . . . 28
7.9 Summary . . . 28

8 Conclusion and Future Work **29**
8.1 Conclusion . . . 29
8.2 Achievement Against Objectives . . . 29
8.3 Main Limitations . . . 30
8.4 Future Work . . . 30
8.4.1 Algorithmic Extensions . . . 30
8.4.2 Fault-Tolerance Extensions . . . 30
8.4.3 Interface and Accessibility Improvements . . . 30
8.4.4 User Study and Stronger Educational Validation . . . 31
8.5 Final Reflection . . . 31

A User Manual **34**
A.1 Running the Tool Locally . . . 34
A.2 Basic Usage . . . 34
A.3 Fault Injection . . . 34
A.4 Scripted Demos . . . 34
A.5 Evidence Export . . . 34

B Test Cases Summary **35**
B.1 Token Ring . . . 35
B.2 Ricart–Agrawala . . . 35

List of Figures

4.1 Main user interface of the Distributed Mutual Exclusion Explorer. 14

6.1 Token Ring: token loss halts progress until regeneration is performed. 20
6.2 Token Ring: crash and recover demonstration. 20
6.3 RA scripted tie-break case: equal timestamps are resolved deterministically using numeric PID ordering. 21
6.4 RA stalled state after dropping an in-flight message; see the trace excerpt below for the missing REPLY dependency. 21
6.5 RA stalled state after dropping the next outgoing message; see the trace excerpt below for the missing REPLY dependency. 22

List of Tables

2.1 Comparison of mutual exclusion algorithm styles relevant to this project. 4

3.1 Requirements-to-design traceability. 8

5.1 Manual test execution summary for the v0.3.2 freeze baseline. 17
5.2 Quantitative testing snapshot for the v0.3.2 baseline. 17

6.1 Summary of key Nielsen heuristic findings (severity scale 0–4). 23

Chapter 1

Introduction

1.1 Motivation

Distributed systems underpin modern computing, but their behaviour is often difficult to understand from static material because correctness depends on concurrency, message ordering, and assumptions about failures [1, 2, 3]. Mutual exclusion is a classic example: a set of distributed processes must coordinate access to a shared critical section without relying on shared memory or a global clock.

Learners commonly struggle to connect algorithm descriptions to concrete executions. This challenge becomes more severe when faults are introduced, since safety properties may remain intact even when progress is lost. Such distinctions are difficult to communicate through static diagrams alone. This motivates the development of an interactive teaching tool that makes state, causality, and fault consequences directly observable.

1.2 Project Aim and Objectives

The aim of this project is to develop an interactive educational artefact for visualising distributed mutual exclusion and its behaviour under faults and recovery conditions.

The specific objectives are:

- implement a technically correct simulation that preserves the safety invariant of mutual exclusion,
- provide step-by-step execution with an explanatory trace,
- support representative token-based and message-based algorithms,
- support fault injection and recovery to illustrate safety vs liveness trade-offs,
- provide reproducible evidence export,
- and evaluate the artefact in terms of correctness, usability, and reproducibility.

1.3 Scope

The delivered system focuses on two representative approaches:

- **Token Ring**, a token-based mechanism that provides an intuitive model of exclusivity.
- **Ricart–Agrawala (RA)**, a message-based algorithm using Lamport-style timestamps [4, 5].

The system is teaching-oriented rather than production-oriented. The network model is intentionally simplified to support controlled observation and reproducibility. Message loss in RA is used to demonstrate that liveness can fail while safety is preserved; retransmission and failure detectors are not implemented.

1.4 Contributions

The main contributions of the project are:

- a browser-based, dependency-free Distributed Mutual Exclusion Explorer,
- support for both Token Ring and an RA teaching-oriented prototype,
- explicit fault injection and recovery mechanisms,
- scripted demos for deterministic replay,
- an evidence export pipeline (state JSON, trace TXT, preview PNG),
- and a reproducible testing and evidence framework built around a freeze baseline.

1.5 Report Structure

Chapter 2 reviews distributed mutual exclusion, logical time, fault assumptions, and educational visualisation literature. Chapter 3 presents the specification and design of the artefact. Chapter 4 describes implementation details. Chapter 5 covers testing strategy and execution. Chapter 6 evaluates correctness, usability, and reproducibility. Chapter 7 discusses professional issues. Chapter 8 concludes the report and outlines future work.

Chapter 2

Background and Related Work

2.1 Distributed Mutual Exclusion

Distributed mutual exclusion requires a set of processes to coordinate access to a critical section using message passing rather than shared memory. Two correctness dimensions are central:

- **Safety:** at most one process may enter the critical section at any time.
- **Liveness:** requesting processes should eventually make progress under suitable assumptions.

This safety–liveness distinction is fundamental in distributed systems and forms the theoretical basis of this project [3, 6, 7].

Mutual exclusion algorithms can be grouped into message-based, token-based, and hierarchical or tree-based families. Each family makes different assumptions and offers different teaching affordances. This project intentionally implements one token-based approach and one message-based approach in order to provide contrast rather than breadth.

2.2 Message-Based and Token-Based Approaches

Message-based algorithms coordinate access through explicit REQUEST/REPLY exchanges. Ricart–Agrawala (RA) is one of the classical examples, requiring each requesting process to receive permission from all peers before entering the critical section [4]. Maekawa's algorithm reduces the number of messages through quorums, but introduces additional conceptual complexity in explaining quorum intersections and deadlock handling [8].

Token-based algorithms instead rely on possession of a unique token. This makes the safety mechanism conceptually simple: a process may enter the critical section only when it holds the token. Suzuki–Kasami is a well-known token-based algorithm for distributed mutual exclusion [9], while ring-based token circulation offers an even simpler mental model for teaching. Tree-based approaches such as Raymond's algorithm provide a more efficient structure but are harder to explain clearly to learners encountering the topic for the first time [10].

Table 2.1 summarises the main approaches relevant to this project.

This comparison also explains the algorithm choices made in the artefact. Token Ring was selected because its exclusivity mechanism is visually and conceptually direct. Ricart–Agrawala was selected because it exposes message ordering, reply dependencies, and the distinction between safety and liveness particularly clearly. Other approaches such as Maekawa and Raymond are valuable, but were not implemented because the project prioritises teaching clarity over algorithmic breadth.

Approach	Representative algorithm	Coordination style	Teaching suitability / relevance here
Token-based	Token Ring / Suzuki–Kasami	Exclusive token ownership	Strong for introducing safety and simple progress reasoning; well suited to visualising token circulation and recovery [9].
Message-based	Ricart–Agrawala	REQUEST/REPLY to all peers	Strong for illustrating ordering, permission dependencies, and safety vs liveness under message loss [4].
Quorum-based	Maekawa	Requests to voting subsets	Interesting but less suitable here because quorum logic adds conceptual overhead [8].
Tree-based	Raymond	Directed token routing in a tree	Useful for advanced comparison, but structurally more complex than needed for a teaching-first prototype [10].

Table 2.1: Comparison of mutual exclusion algorithm styles relevant to this project.

2.3 Logical Time and Ordering

In distributed systems there is no global clock that can be relied upon for event ordering. Lamport's logical clocks provide a way to establish a partial order of events through message causality [5]. This idea is essential for message-based coordination algorithms, because processes must decide whether to reply immediately or defer a reply according to the relative priority of requests.

For teaching purposes, logical time is one of the most abstract parts of distributed systems. A visual tool can make this more concrete by showing:

- when a request is broadcast,
- when a reply is received,
- and how ties are resolved deterministically.

This is one reason Ricart–Agrawala is particularly suitable for an educational artefact: its behaviour depends directly on ordering rules that can be surfaced step-by-step.

2.4 Failures, Assumptions, and the Safety–Liveness Distinction

Distributed algorithms cannot be evaluated independently of their assumptions. Message loss, crash failures, and timing uncertainty can all alter whether a requesting process eventually enters the critical section. More broadly, classical results such as FLP demonstrate that guarantees depend strongly on the failure and timing model [11]. In practice, crash failures and message loss may preserve safety while breaking liveness unless the system provides additional recovery mechanisms such as retransmission, failure detectors, or stronger synchrony assumptions [12].

This point is particularly important pedagogically. Students often interpret "the algorithm did not complete" as "the implementation is wrong", when in fact the correct conclusion may be that the assumptions needed for liveness were violated. The present project uses explicit fault controls and trace explanations so that this distinction becomes visible.

2.5 Educational Visualisation and Interactive Learning

Research on algorithm visualisation shows that animation alone does not guarantee learning; effectiveness depends on interaction, engagement, and the learner's opportunity to interpret what is being shown [13, 14]. Reviews of educational visualisation further suggest that visual tools are most useful when they make hidden state visible and support structured observation rather than passive viewing [15, 16, 17].

The artefact in this project follows those principles in four ways:

- execution is step-based rather than opaque;
- hidden algorithm state is externalised in a trace and tables;
- fault injection allows learners to test assumptions directly;
- scripted scenarios make demonstrations repeatable for discussion and reflection.

2.6 Cognitive Load, Usability, and Tool Design

Complex distributed behaviour can impose substantial intrinsic cognitive load. Cognitive load theory argues that instructional tools should minimise extraneous complexity while supporting schema formation [18]. Multimedia learning principles also emphasise that explanations and representations should be aligned with learner needs [19]. In addition, minimally guided interfaces can be unhelpful for novices unless appropriate scaffolding is present [20].

These ideas motivate several design choices in this project:

- a deterministic step engine,
- concise trace messages,
- explicit stalled-state explanations,
- and grouped controls that separate algorithm choice, fault injection, and export functions.

Usability is later evaluated using Nielsen's heuristic framework [21] and contextualised using ISO 9241-11 [22].

2.7 Reproducibility and Research Software Practice

Reproducibility is increasingly recognised as a requirement in computational research [23, 24, 25]. This includes versioned baselines, explicit execution steps, and evidence that links claims to raw artefacts. In educational tools, reproducibility also has a pedagogical dimension: students should be able to replay the same scenario and observe the same behaviour.

This project therefore treats reproducibility as part of the artefact itself, rather than as an afterthought. Scripted demos, evidence exports, indexed evidence sets, and CI-backed smoke tests all support this aim.

2.8 Gap and Positioning

Existing distributed systems textbooks and algorithm papers explain mutual exclusion algorithms and their theoretical properties well [1, 2, 3, 4, 9]. Existing visualisation and learning literature explains why interactive tools can support understanding [13, 14, 16]. However, there is a gap between these

two bodies of work: there is comparatively little lightweight, browser-based tooling that simultaneously emphasises:

- distributed mutual exclusion as a focused topic,
- explicit fault and recovery scenarios,
- traceable evidence export for reproducibility,
- and a direct teaching of the distinction between safety and liveness.

The present project is positioned to address that gap. It does not claim to invent a new algorithm. Instead, its contribution lies in combining representative algorithms, controlled fault injection, scripted reproducibility, and explanation-oriented feedback in a single teaching artefact.

Chapter 3

Specification and Design

3.1 Development Methodology

The project followed an iterative and incremental development methodology. Early iterations focused on establishing a stable simulation core and a minimal browser-based interface. Later iterations added:

- fault injection and recovery controls,
- scripted scenarios for repeatable demonstrations,
- evidence export,
- and reproducibility features such as CI-backed smoke tests and indexed evidence sets.

A feature freeze baseline was established at `v0.3.2`. This separated artefact development from report writing and evidence curation. The approach can therefore be characterised as **evidence-driven refinement**: once the core features were stable, subsequent changes focused on testing, traceability, and report support rather than introducing new behaviours.

3.2 Requirements Approach

A lightweight but structured requirements approach was adopted, consistent with software requirements guidance [26]. Requirements were derived from the project brief, the educational objective of the artefact, and the need to support clear evaluation.

3.3 Functional Requirements

The main functional requirements are:

- **FR1: Configuration.** Users can select the algorithm and number of processes.
- **FR2: Interactive execution.** Users can request/release the critical section and advance execution using Step, Run, Pause, and Reset.
- **FR3: State visibility.** The tool exposes process state, safety status, and, for RA, the message queue.
- **FR4: Fault handling.** Users can inject representative faults and, where appropriate, recovery actions.
- **FR5: Scripted scenarios.** Users can load deterministic demonstrations for teaching and evidence capture.
- **FR6: Evidence export.** Users can export a state snapshot, trace, and preview image.

3.4 Non-functional Requirements

The principal non-functional requirements are:

- **NFR1: Safety.** Mutual exclusion must be preserved.
- **NFR2: Clarity.** Users should be able to explain why progress occurs or stalls.
- **NFR3: Reproducibility.** Scenarios and evidence must be repeatable.
- **NFR4: Portability.** The artefact should run in a modern browser without external dependencies.
- **NFR5: Maintainability.** The codebase should remain small and modular.

3.5 Architecture Overview

The architecture is divided into two principal layers:

- **Core simulation layer:** maintains algorithm state, defines transitions, checks safety, and handles import/export.
- **UI controller layer:** renders state, handles input, and coordinates user-facing behaviour.

This separation reduces coupling and supports headless automated testing of the core logic [27]. It also allows the UI to be refined without changing the semantics of the algorithms.

3.6 Requirements-to-Design Traceability

To make the design rationale explicit, Table 3.1 maps key requirements to the corresponding design decisions.

Requirement	Design decision	Rationale
Fault demonstration	Token loss, crash/recover, drop-next-send, drop in-flight message	Makes the consequences of violated assumptions directly observable; supports safety vs liveness teaching.
Reproducibility	Scripted demos + evidence export + freeze baseline	Enables report claims to be audited and scenarios to be replayed consistently.
Clarity	Trace + queue table + preview canvas + safety indicator	Reduces hidden state and supports step-by-step explanation.
Maintainability	Separation between core engine and UI controller	Keeps algorithmic behaviour testable and reduces accidental coupling.
Portability	Browser-only, dependency-free implementation	Lowers deployment friction and supports static hosting.

Table 3.1: Requirements-to-design traceability.

3.7 Execution Model

A central design decision is the use of a **step engine**. Each press of `Step` advances the simulation by one minimal unit of progress:

- Token Ring: token movement or critical section transition.
- Ricart–Agrawala: delivery of exactly one queued message.

This is a deliberate design choice rather than an implementation convenience. A queue-based, stepwise execution model is easier to reason about pedagogically than a hidden asynchronous scheduler, because learners can directly observe which action caused which state change.

3.8 Key Design Trade-offs

Several trade-offs were made consciously:

3.8.1 Queue-Based Network Abstraction

The RA network is modelled as a FIFO queue of in-flight messages. This is less realistic than a general asynchronous network model, but more suitable for teaching because:

- one step corresponds to one visible message delivery,
- queue state can be shown in a table,
- and message-loss faults can be injected in a controlled way.

3.8.2 No Retransmission or Failure Detector

The RA implementation does not include retransmission, timeouts, or failure detectors. This is intentional. The project aims to demonstrate that liveness can fail when assumptions are weakened. Adding recovery protocols would make the model more realistic, but would also reduce the clarity of the core teaching point [12].

3.8.3 Browser-Only and Dependency-Free

A browser-only artefact was chosen for accessibility, deployment simplicity, and sustainability. This reduces installation overhead and makes the tool easier to reproduce and distribute.

3.9 Reproducibility by Design

Reproducibility is built into the design through:

- a freeze baseline tag and release,
- scripted demos,
- evidence triplet export,
- indexed evidence and figure mappings,
- and CI-backed smoke tests.

This ensures that evaluation is not just descriptive, but backed by re-runnable artefacts and documented procedures [23, 24, 25].

3.10 Design Summary

The design of the artefact is guided by a simple principle: *make distributed mutual exclusion visible, explainable, and reproducible*. The resulting system is intentionally modest in algorithmic breadth, but strong in clarity, traceability, and suitability for evaluation.

Chapter 4

Implementation

4.1 Implementation Goals

The delivered artefact is a dependency-free browser application designed for interactive exploration of distributed mutual exclusion. The implementation prioritises correctness (especially the safety invariant), pedagogical clarity, and reproducibility. Two representative approaches are implemented:

- a token-based **Token Ring** model;
- a message-based **Ricart–Agrawala (RA)** teaching-oriented prototype [4, 5].

4.2 Repository Structure and Responsibilities

The codebase is organised around a small number of responsibilities:

- `index.html`: page layout and controls.
- `styles.css`: minimal CSS for layout and readability.
- `mutex_core.js`: algorithm models, state transitions, safety checking, script replay, import/export.
- `mutex_main.js`: DOM wiring, rendering, preview drawing, evidence export, and interaction logic.
- `examples/`: scripted JSON scenarios.
- `tools/smoke_tests.mjs`: dependency-free automated smoke tests.

This separation keeps the simulation engine testable without a browser and prevents UI changes from implicitly changing algorithm behaviour [27].

4.3 Core Data Structures

A single `model` object captures the current simulator state. It stores:

- the selected algorithm and current mode,
- the list of processes,
- algorithm-specific state (token information or message queue),
- a step-numbered trace,
- and lightweight metrics.

4.3.1 Process State

Each process includes:

- `id`

- `requesting`
- `inCS`
- `crashed`

For RA, additional fields include:

- `clock`
- `reqTs`
- `awaiting`
- `deferred`

4.3.2 Token Ring State

Token Ring uses:

- `ring`
- `token.holder`
- `token.lost`

4.3.3 RA Network State

The RA prototype uses:

- `network.queue`
- `network.nextMsgId`
- `network.dropNextSend`

4.3.4 Trace and Metrics

A step-numbered trace records user actions, algorithm events, warnings, and stall explanations. Metrics record token passes, CS entries, releases, and for RA the number of messages sent, delivered, and dropped.

4.4 Safety Checking

The safety property is encoded directly in the core module through a dedicated checker. The invariant is:

> At most one process may be in the critical section at any time.

The UI exposes this as a Safety indicator, making the invariant visible during every scenario.

4.5 Step Engine

A core design feature is the **step engine**. Each press of `Step` performs one minimal unit of progress.

4.5.1 Token Ring Step Semantics

A Token Ring step follows this decision order:

1. If the token is lost, report that progress cannot continue.

2. If a process is already in the critical section, report that release is required.
3. If the token holder is requesting, allow entry to the critical section.
4. Otherwise, pass the token to the next process in ring order.

4.5.2 RA Step Semantics

An RA step follows this decision order:

1. If the network queue is non-empty, deliver exactly one queued message.
2. If the queue is empty but a process is still awaiting replies, emit a stall explanation.
3. Otherwise, report that no messages are in flight.

4.6 Token Ring Implementation

Token Ring provides mutual exclusion through token ownership. A process requests the critical section by setting `requesting = true`. If the token holder is requesting and no process is already in the critical section, it enters.

4.6.1 Token Faults and Recovery

Token Ring supports:

- **Drop token**: sets `token.lost = true`, halting progress.
- **Regenerate token**: restores the token.
- **Crash / recover**: makes a process unavailable and later returns it to participation.

4.7 Ricart–Agrawala Implementation

In the RA prototype, a requesting process:

1. increments its logical clock,
2. stores the request timestamp,
3. initialises its outstanding reply set,
4. and enqueues REQUEST messages to all peers.

When a REQUEST is delivered, the receiver either:

- replies immediately, or
- defers reply if it is already in the critical section or has priority.

Entry occurs only after all required REPLY messages are received.

4.7.1 Tie-break Logic

When requests share the same timestamp, the implementation resolves the tie using numeric process IDs, ensuring deterministic and reproducible behaviour.

4.8 Fault Injection Points

The fault controls are implemented in the simulation engine.

4.8.1 Token Ring

- `dropToken(model)`
- `regenerateToken(model)`
- `crashProcess(model, pid)`
- `recoverProcess(model, pid)`

4.8.2 Ricart–Agrawala

- `dropNextMessage(model)`
- `toggleDropNextSend(model)`
- crash/recover controls

4.9 Stall Explanation Logic

A key clarity feature is explicit stalled-state explanation. In RA, if the queue is empty but a process still awaits replies, the simulator emits a trace message indicating which REPLY is missing. This prevents learners from misinterpreting "queue empty" as successful completion.

4.10 Scripted Scenarios

Scripted scenarios are stored as JSON files and replayed via the same step engine used in interactive mode. This supports repeatable demonstrations and reproducible evidence capture.

4.11 Export Pipeline

The tool exports:

- state JSON,
- trace TXT,
- preview PNG.

The export naming strategy combines a timestamp-based session identifier, algorithm/mode metadata, and a sequence number, producing stable and traceable evidence artefacts.

4.12 Implementation Limitations

The implementation intentionally simplifies some aspects:

- the RA network is modelled as a FIFO queue,
- no retransmission or failure detector is implemented,
- the visual design prioritises clarity over animation.

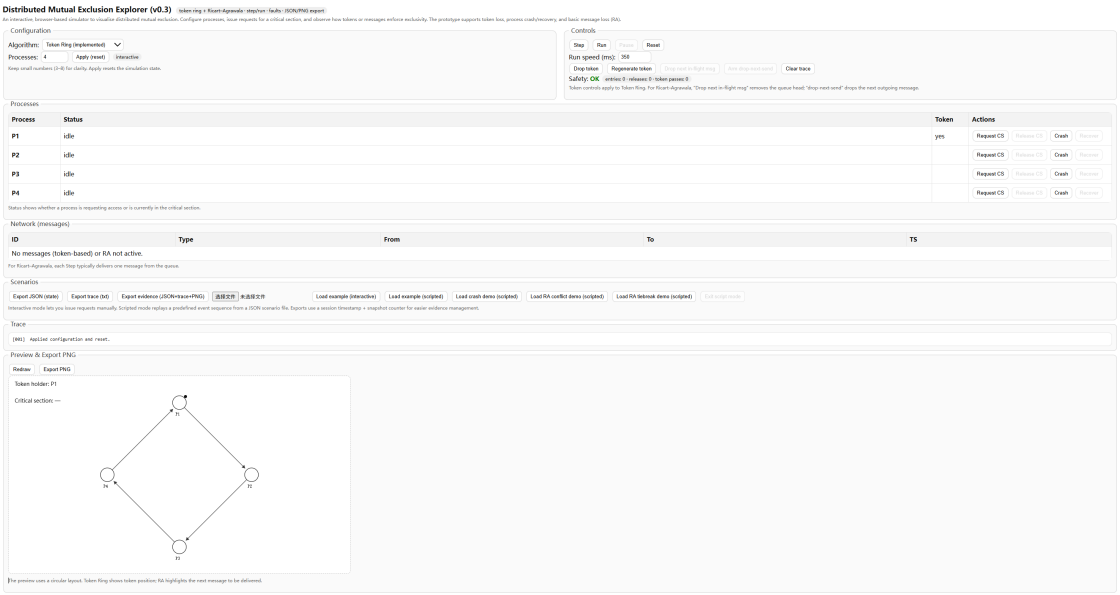

Figure 4.1: *Main user interface of the Distributed Mutual Exclusion Explorer.*

Chapter 5

Testing

5.1 Testing Methodology

The testing methodology combines two complementary components:

- **Manual evidence-driven tests** validate UI behaviour, scripted scenarios, and fault demonstrations as experienced by a learner.
- **Automated smoke and unit tests** validate the core simulation engine independently of the browser environment.

Both are necessary because the artefact is interactive. Automated tests are strong for preserving core logic and preventing regressions, whereas manual tests are better suited to validating trace readability, scenario flow, and evidence capture.

5.2 Testing Objectives

Testing aims to demonstrate that the artefact:

1. preserves the mutual exclusion safety invariant,
2. exhibits intended behaviour under normal conditions,
3. demonstrates fault effects and recoveries in a controlled way,
4. and produces reproducible evidence for later evaluation.

5.3 Manual Evidence-Driven Testing

A structured manual test plan is maintained in `eval/test_plan.md`. The minimum executed set for the freeze baseline includes:

- **Token Ring:**
  - TR-01: basic mutual exclusion,
  - TR-02: token loss and regeneration,
  - TR-03: crash and recover.
- **Ricart–Agrawala:**
  - RA-01: basic request/entry/release,
  - RA-02: tie-break under equal timestamps,
  - RA-03: drop next in-flight message causing a stalled state,
  - RA-04: drop-next-send causing a stalled state.

Each manual case produces an evidence triplet:

- state JSON,
- trace TXT,
- preview PNG.

This evidence-first approach supports later evaluation because every behavioural claim can be tied to a stored artefact.

5.4 Automated Testing

Automated tests are implemented using the Node.js built-in test runner and are executed through:

```
npm test
```

The automated workflow includes:

- smoke tests for quick regression checks,
- unit tests for the core simulation logic,
- and coverage reporting for the core module.

These tests target `mutex_core.js`, which contains the main algorithmic behaviour. This choice is deliberate: the most critical regressions for this project are algorithmic, not cosmetic.

5.5 Continuous Integration

A GitHub Actions workflow runs the automated suite on each push and pull request. This provides:

- rapid detection of regressions,
- a clean execution environment,
- and an auditable history of results [28].

5.6 Pass Criteria and Outcome Categories

Testing is not limited to binary success/failure. Two result categories are used:

- **PASS:** the observed behaviour matches the expected correctness or recovery behaviour.
- **Expected stall:** the artefact intentionally demonstrates a liveness failure under fault assumptions while preserving safety.

This distinction is particularly important for RA-03 and RA-04, where message loss is expected to block progress.

5.7 Test Execution Summary

Table 5.1 summarises the key manual cases and their outcomes.

5.8 Quantitative Snapshot

The testing process also yields quantitative evidence. Table 5.2 summarises the automated results and evidence footprint.

Case ID	Mode	Fault injected	Pass criterion	Result	Evidence pointer
TR-01	interactive	none	At most one process may be in the critical section.	PASS	Evidence case TR-01
TR-02	interactive	token loss + regenerate	No progress until regeneration; progress resumes after token regeneration.	PASS	Evidence case TR-02
TR-03	scripted / interactive	crash + recover	Crash and recovery are visible; progress resumes after recovery.	PASS	Evidence case TR-03
RA-01	interactive	none	Entry occurs only after all required REPLY messages are received.	PASS	Evidence case RA-01
RA-02	scripted	none	Deterministic tie-break under equal timestamps.	PASS	Evidence case RA-02
RA-03	interactive	drop next in-flight msg	Explicit stall explanation is shown while safety remains preserved.	Expected stall	Evidence case RA-03
RA-04	interactive	drop-next-send	Explicit stall explanation is shown while safety remains preserved.	Expected stall	Evidence case RA-04

Table 5.1: Manual test execution summary for the v0.3.2 freeze baseline.

Metric	Value
Smoke tests passed	6 / 6
Unit tests passed	31 / 31
Core line coverage (`mutex_core.js`)	96.28%
Core branch coverage (`mutex_core.js`)	83.18%
Core function coverage (`mutex_core.js`)	98.39%
Manual evidence cases executed	7
Curated report figures selected	5

Table 5.2: Quantitative testing snapshot for the v0.3.2 baseline.

5.9 Evidence Capture Workflow

For each manual case, the following workflow was used:

1. execute the scenario from the test plan,
2. export the evidence triplet,
3. store raw artefacts in the corresponding evidence case folder,
4. curate selected figures for the report,
5. update the evidence and figure indices.

This provides a direct path from scenario execution to report evidence.

5.10 Testing Limitations

The testing strategy has limitations:

- browser-level UI automation is not implemented,
- some edge cases are more meaningfully validated through manual evidence than through automation,
- and RA liveness under message loss is intentionally demonstrated rather than repaired.

These limitations are acceptable given the scope of an educational artefact and are made explicit to avoid over-claiming.

Chapter 6

Evaluation

6.1 Evaluation Methodology

The evaluation methodology is designed to answer four questions:

1. Does the artefact preserve mutual exclusion safety?
2. Do the implemented fault scenarios produce meaningful and explainable behaviour?
3. Is the tool usable enough for learning-oriented exploration?
4. Are the results reproducible and traceable?

To answer these questions, the evaluation draws on four data sources:

- manual evidence-driven test cases,
- exported evidence triplets (JSON/trace/PNG),
- a Nielsen heuristic evaluation,
- and CI-backed automated test outputs.

The methodology therefore combines behavioural validation, usability inspection, and reproducibility analysis rather than relying on a single metric.

6.2 Evaluation Criteria

Four criteria are used:

- **Correctness:** preservation of the mutual exclusion safety invariant.
- **Fault clarity:** faults should produce understandable and meaningful effects.
- **Usability and pedagogical clarity:** users should be able to interpret state, progress, and stalled conditions.
- **Reproducibility:** evidence should be regenerable from a fixed baseline.

6.3 Correctness and Safety

Across both Token Ring and RA scenarios, the central safety invariant is preserved: at most one process may be in the critical section at any time [3, 6]. This invariant is visible through the Safety indicator and was not violated in any manual or automated test case.

For Token Ring, the interpretation is straightforward: only the token holder may enter the critical section. For RA, entry requires collecting all necessary REPLY messages. In both cases, the tool's execution and trace align with the expected safety model.

6.3.1 Token Ring Fault Cases

Token loss and crash/recover demonstrate that safety can remain intact while progress becomes impossible until recovery occurs. This supports the interpretation that correct distributed behaviour depends not only on exclusivity but also on system assumptions about resource availability.

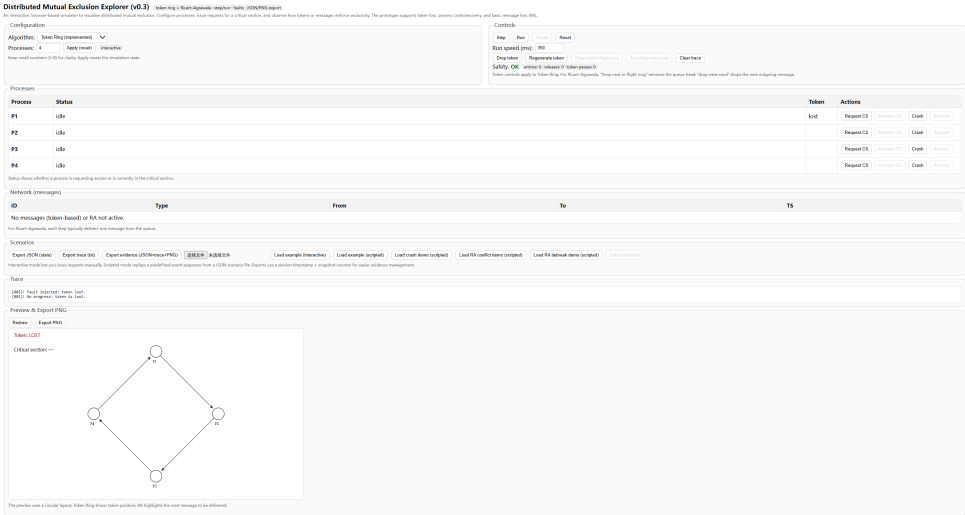

Figure 6.1: Token Ring: token loss halts progress until regeneration is performed.

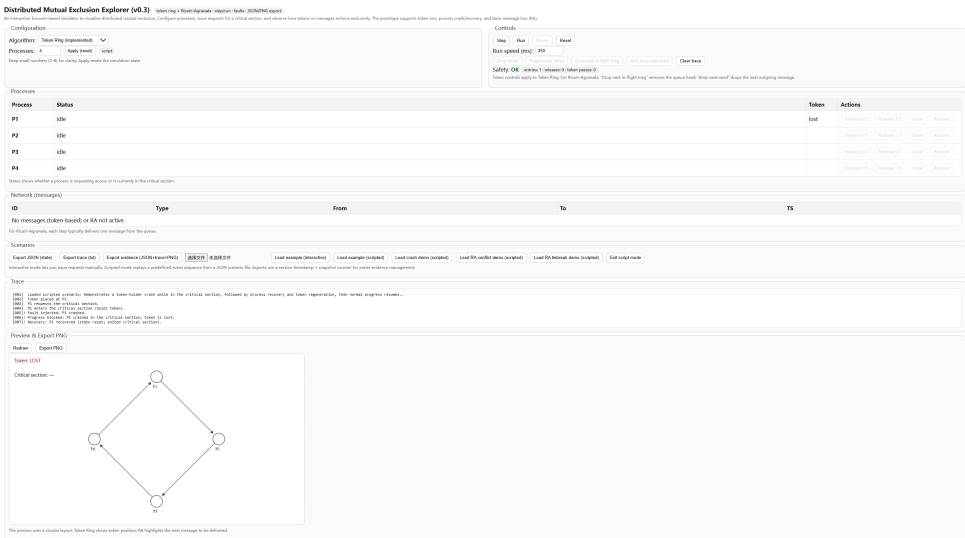

Figure 6.2: Token Ring: crash and recover demonstration.

6.3.2 RA Tie-break Behaviour

A scripted tie-break scenario demonstrates deterministic ordering when requests share the same logical timestamp. Numeric PID ordering is used to resolve the tie, ensuring a stable and explainable outcome [5, 4].

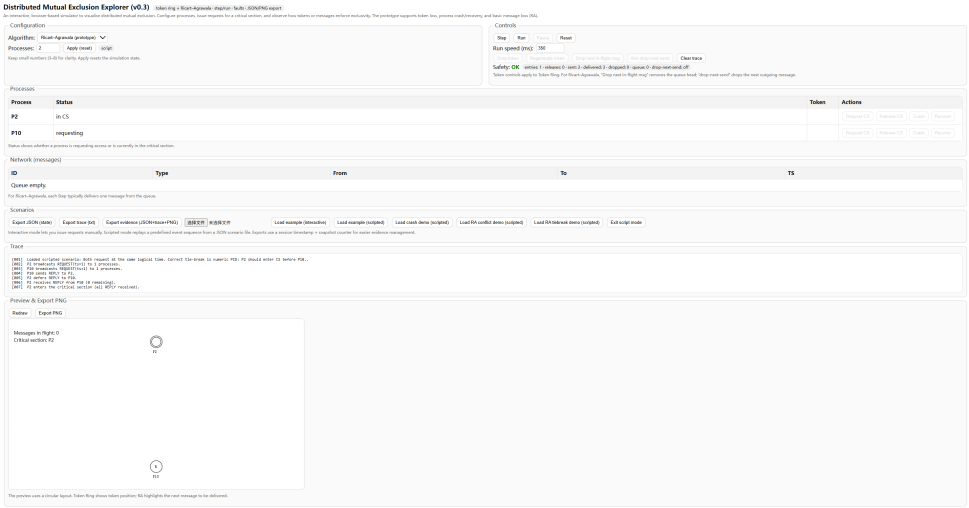

Figure 6.3: RA scripted tie-break case: equal timestamps are resolved deterministically using numeric PID ordering.

6.4 Fault Demonstrations: Safety vs Liveness

The most pedagogically significant result is the explicit separation between safety and liveness. In the RA prototype, message loss can prevent a process from collecting all required REPLY messages, so progress may halt while mutual exclusion remains preserved [3].

6.4.1 RA-03: Drop Next In-Flight Message

RA-03 demonstrates network-side message loss. Figure 6.4 shows the stalled global state after dropping an in-flight message. The figure shows the overall state of the system, while the trace excerpt below identifies the specific missing dependency.

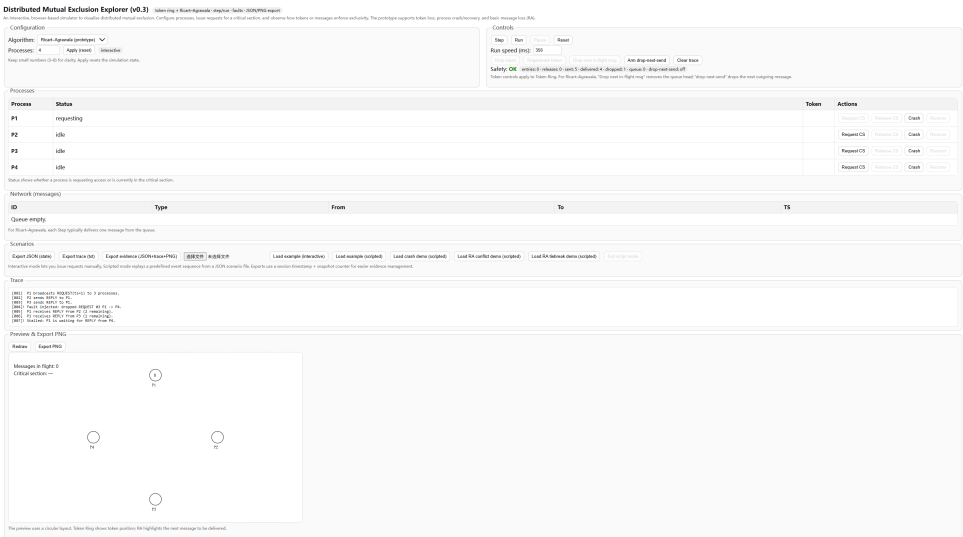

Figure 6.4: RA stalled state after dropping an in-flight message; see the trace excerpt below for the missing REPLY dependency.

```
Fault injected: dropped REQUEST #3 P1 -> P4.
```

```
Stalled: P1 is waiting for REPLY from P4.
```

This case demonstrates that progress can fail without violating mutual exclusion. The artefact therefore helps learners separate the notion of a correct safety property from the distinct question of eventual progress.

6.4.2 RA-04: Drop-Next-Send

RA-04 demonstrates send-side message loss. The resulting global stalled state is shown in Figure 6.5, and the trace excerpt below provides the causal explanation.

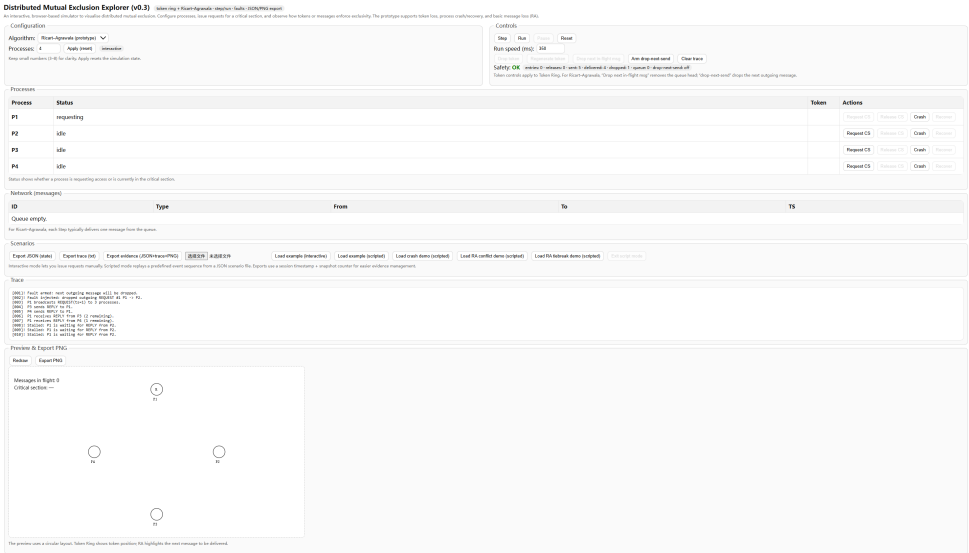

Figure 6.5: RA stalled state after dropping the next outgoing message; see the trace excerpt below for the missing REPLY dependency.

```
Fault armed: next outgoing message will be dropped.
Fault injected: dropped outgoing REQUEST #1 P1 -> P2.
Stalled: P1 is waiting for REPLY from P2.
```

Together, RA-03 and RA-04 demonstrate that the artefact is not only useful for showing correct runs, but also for making fault consequences and dependency failures visible.

6.5 Usability Evaluation

Usability was assessed using Nielsen's heuristic framework [21]. The updated checklist covers:

- algorithm selection,
- fault controls,
- queue visibility,
- trace readability,
- and evidence export.

Table 6.1 summarises the most relevant findings and severity ratings.

Finding	Severity	Interpretation / Action
Trace clearly explains state changes and stalls	1	Strong learning support; retain as a core feature.
Preview symbols may require a legend for new users	2	Add a small legend in a future iteration.
Fault controls are useful but visually dense	1	Current grouping is acceptable; optional advanced toggle later.
Algorithm differences are not explained in-app	2	Add concise algorithm help text later.
Evidence export is useful but not fully self-explanatory to first-time users	1	Covered by README and appendices; in-app guidance could be improved.

Table 6.1: Summary of key Nielsen heuristic findings (severity scale 0–4).

Overall, the artefact performs well on visibility of system status, user control, and fault explainability. Remaining usability work is concentrated in discoverability and lightweight in-app guidance rather than in the correctness of interaction.

6.6 Reproducibility

Reproducibility is a major strength of the project. It is supported by:

- a freeze baseline tag and release,
- CI-backed automated tests,
- evidence triplet exports,
- evidence and figure indices,
- and a consistent local run method.

A compact reproducibility protocol is:

1. checkout tag `v0.3.2`,
2. run `python -m http.server 5500`,
3. open the local URL,
4. load the relevant demo or perform the manual case steps,
5. export the evidence triplet,
6. compare the exported trace and preview against the report references.

This turns reproducibility into an executable process rather than a rhetorical claim [23, 24, 25].

6.7 Threats to Validity

The evaluation has several limitations:

- the queue-based network is a teaching abstraction rather than a high-fidelity asynchronous model,
- RA is intentionally presented as a teaching-oriented prototype rather than a full fault-tolerant protocol,
- usability was evaluated heuristically rather than through a substantial participant study.

These limitations are documented explicitly to avoid overstating the scope of the conclusions.

6.8 Summary

The evaluation shows that the artefact meets its educational objectives:

- safety is preserved across tested scenarios,
- fault injection successfully illustrates progress and liveness trade-offs,
- usability is strong in visibility and control,
- and reproducible evidence can be generated and audited.

The combined result is not merely a working simulator, but a reportable and inspectable educational artefact.

Chapter 7

Professional Issues

7.1 Overview

Final-year computing projects are expected to address not only technical correctness, but also the broader professional issues that shape responsible software development. For this project, the most relevant issues are:

- accessibility,
- privacy and data handling,
- sustainability and maintainability,
- reproducibility and research integrity,
- and ethics in evaluation and communication.

These issues are particularly important because the artefact is intended as an educational tool: its value depends not only on whether it works, but also on whether it can be used, trusted, maintained, and interpreted responsibly.

7.2 Accessibility

Accessibility is an important professional responsibility for educational software. A tool designed to support learning should not unnecessarily exclude users because of avoidable interface barriers. Current accessibility-relevant strengths of the artefact include:

- explicit textual status indicators (for example, process states, the Safety indicator, and trace messages),
- grouped controls with consistent action labels,
- and the use of multiple views (tables, trace, preview) so that critical information is not conveyed only through a single visual channel.

These design choices are helpful, but they do not by themselves guarantee accessibility. Relative to accessibility guidance such as WCAG 2.2 and ARIA recommendations [29, 30], several limitations remain:

- the interface has not been systematically evaluated for keyboard-only navigation,
- screen-reader behaviour has not been formally tested,
- and some visual markers in the preview (for example, the request marker, token indicator, and crash marker) may still benefit from a compact legend or additional textual equivalents.

Accordingly, the current position of the project is that accessibility has been considered at a basic practical level, but there remains clear scope for improvement in a future iteration. This is acceptable for a prototype, provided the limitation is stated explicitly.

7.3 Privacy and Data Handling

The privacy implications of the artefact are limited, because the simulator is designed to run locally or as a static web page and does not require user accounts or server-side storage. By default:

- no personal data are collected,
- no analytics are embedded,
- and exported JSON/trace/PNG files contain only simulation state and evidence data.

This is a positive design choice because it reduces risk and simplifies both deployment and explanation. However, privacy still matters at the level of workflow and reporting. If screenshots, traces, or future participant feedback are collected, these materials should be managed carefully and stored only where necessary. In the current project, all evidence is technical rather than personal, which keeps privacy concerns low.

7.4 Ethics and Research Governance

No formal human-subject study was conducted as part of the current evaluation. The evaluation therefore remains within a low-risk scope based on:

- manual evidence-driven testing,
- heuristic usability inspection,
- and automated testing of the artefact itself.

This is an important point for research governance: the current report does not make claims based on participant data, and therefore does not present the artefact as having been validated through a controlled human study.

If the project were extended to include participant-based evaluation (for example, a structured comparison of learner performance or a SUS questionnaire), appropriate ethical safeguards would be required. These would include:

- informed consent,
- data minimisation,
- anonymisation where appropriate,
- and compliance with the university's relevant ethics procedures.

Stating this boundary clearly avoids over-claiming the scope of the present evaluation.

7.5 Sustainability and Maintainability

A major professional consideration for software artefacts is whether they remain maintainable over time. In this project, sustainability was addressed through several practical engineering choices:

- the artefact is dependency-free and browser-based,
- the codebase is intentionally small,
- the core simulation engine is separated from the UI layer,
- and version control plus tagged releases provide traceability.

These choices reduce the likelihood of build breakage due to external dependency drift and make

the system easier to understand and evolve. In this sense, the project adopts a deliberately conservative architecture in order to improve long-term robustness [28, 24].

At the same time, maintainability is not only a question of code structure. It also depends on whether decisions are documented and whether testing protects against regressions. The inclusion of smoke tests, unit tests, evidence indices, and release notes therefore contributes directly to maintainability.

7.6 Reproducibility and Research Integrity

Reproducibility is one of the strongest professional aspects of the project. Rather than treating reproducibility as an optional afterthought, the project integrates it into the artefact workflow itself. Specifically:

- the project defines a freeze baseline release,
- evidence is exported as a triplet (state JSON, trace TXT, preview PNG),
- evidence indices and figure indices map report claims to raw artefacts,
- and CI-backed smoke tests provide automated regression checks.

This supports research integrity in two ways. First, it makes the evaluation claims auditable. Second, it reduces the risk that report figures and conclusions become detached from the actual implementation [23, 24, 25].

A related issue is honest communication of uncertainty. The report explicitly distinguishes between:

- what is demonstrated empirically,
- what is inferred from those demonstrations,
- and what remains out of scope.

For example, the Ricart–Agrawala implementation is described as a teaching-oriented prototype rather than a fully fault-tolerant production protocol. This is not merely a wording choice; it is part of maintaining professional and academic integrity.

7.7 Responsible Communication of Limitations

A professional report should make limitations clear rather than hide them. This project has several important limitations:

- the network is modelled as a queue-based teaching abstraction,
- retransmission and failure detectors are not implemented,
- usability evaluation is heuristic rather than participant-based,
- and some accessibility aspects remain future work.

These limitations are not failures of the project; rather, they are part of its scope definition. The responsible approach is to explain them clearly and to avoid making claims that would imply a stronger validation or a more realistic distributed environment than the project actually provides.

7.8 Professional Standards and Evaluation Practice

Usability discussion in this report is informed by Nielsen's heuristics [21] and broader usability concepts from ISO 9241-11 [22]. These frameworks help anchor the evaluation in recognised professional practice. Similarly, reproducibility and version-control choices align the project with established norms in research software development [23, 24, 28].

7.9 Summary

The professional issues analysis shows that the project is not only technically competent, but also developed with attention to responsible software practice. Its main strengths are low privacy risk, strong reproducibility, and a maintainable architecture. Its main limitations lie in accessibility completeness and the absence of participant-based evaluation. These are acceptable for the current scope, provided they are documented transparently and treated as priorities for future work.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This project set out to develop an interactive educational artefact for learning distributed mutual exclusion. The resulting system, the *Distributed Mutual Exclusion Explorer*, provides a browser-based environment in which learners can observe and manipulate two representative coordination approaches:

- a token-based Token Ring model,
- and a message-based Ricart–Agrawala (RA) teaching-oriented prototype.

The central achievement of the project is not the invention of a new algorithm, but the creation of a tool that makes the behaviour of distributed mutual exclusion visible, explainable, and reproducible. In particular, the artefact supports:

- step-by-step execution,
- scripted demos,
- fault injection and recovery,
- trace-based explanations,
- and exportable evidence suitable for report-based evaluation.

From a correctness perspective, the project demonstrates the preservation of the mutual exclusion safety invariant across the tested scenarios. From an educational perspective, the artefact is especially strong in making the distinction between *safety* and *liveness* concrete. This is most visible in the RA message-loss cases, where the tool does not simply fail silently, but instead explains why progress has stalled.

The project also makes a contribution at the level of engineering and reproducibility. By combining:

- a freeze baseline release,
- automated smoke and unit testing,
- evidence triplet export,
- and indexed mappings between figures and evidence,

the project supports a report workflow in which claims are traceable back to concrete artefacts. This strengthens the credibility of the written evaluation and aligns the project with professional software and research practices.

8.2 Achievement Against Objectives

The objectives stated in Chapter 1 have been substantially met.

- A technically correct simulation preserving the mutual exclusion safety property was implemented.
- Step-by-step execution and explanatory traces were provided for both algorithms.

- Token-based and message-based mutual exclusion were both represented.
- Fault injection and recovery mechanisms were added and evaluated.
- Reproducible evidence export and indexing were integrated into the workflow.
- Correctness, usability, and reproducibility were evaluated through structured methods.

Taken together, these outcomes mean that the project satisfies the main educational and technical aims of Variant 3.

8.3 Main Limitations

Several limitations remain and should be recognised explicitly.

First, the RA component is intentionally a *teaching-oriented prototype*. It does not attempt to model all aspects of realistic distributed communication, such as retransmission, timeouts, or failure detectors. Second, the network representation is deliberately simplified into a queue-based model to support visibility and reproducibility. Third, usability was assessed heuristically rather than through a large participant study. Finally, while accessibility has been considered, it has not yet been evaluated systematically.

These limitations do not invalidate the project, but they define its scope and should guide how its contributions are interpreted.

8.4 Future Work

Several directions for future work follow naturally from the current artefact.

8.4.1 Algorithmic Extensions

The current system focuses on Token Ring and RA because they provide a strong contrast between token-based and message-based coordination. Future work could add:

- Suzuki–Kasami for a richer token-based comparison [9],
- Maekawa to illustrate quorum-based trade-offs [8],
- or Raymond's algorithm for a more structured token-routing approach [10].

8.4.2 Fault-Tolerance Extensions

For RA in particular, a natural next step would be to add optional recovery-oriented mechanisms such as:

- timeouts,
- retransmissions,
- or simplified failure-detector logic [12].

This would allow the tool to contrast *pure algorithmic assumptions* with *fault-tolerant engineering extensions*.

8.4.3 Interface and Accessibility Improvements

Several usability and accessibility improvements are also realistic:

- a legend for preview symbols,
- concise in-app algorithm explanations,
- keyboard shortcuts,
- and stronger keyboard/screen-reader support aligned with WCAG and ARIA guidance [29, 30].

8.4.4 User Study and Stronger Educational Validation

A future iteration could extend the current heuristic evaluation with a participant-based study. This might include:

- structured learner tasks,
- pre-/post-task conceptual questions,
- or a lightweight usability instrument such as SUS [31].

This would strengthen claims about pedagogical effectiveness beyond the current evidence-driven and heuristic framework.

8.5 Final Reflection

The final outcome of the project is a working and evaluable teaching artefact that makes an abstract area of distributed systems more concrete. It demonstrates that meaningful final-year project work in computing is not limited to building a technically functioning system; it also involves creating something that can be reasoned about, tested, evaluated, and communicated responsibly. In that sense, the project succeeds both as a software artefact and as a piece of engineering-led academic work.

Bibliography

[1] Andrew S. Tanenbaum and Maarten van Steen. *Distributed Systems*. Pearson, 3 edition, 2017.

[2] George Coulouris, Jean Dollimore, Tim Kindberg, and Gordon Blair. *Distributed Systems: Concepts and Design*. Addison-Wesley, 5 edition, 2011.

[3] Nancy A. Lynch. *Distributed Algorithms*. Morgan Kaufmann, 1996.

[4] Glenn Ricart and Ashok K. Agrawala. An optimal algorithm for mutual exclusion in computer networks. *Communications of the ACM*, 24(1):9–17, 1981.

[5] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

[6] Hagit Attiya and Jennifer Welch. *Distributed Computing: Fundamentals, Simulations, and Advanced Topics*. Wiley, 2 edition, 2004.

[7] Gerard Tel. *Introduction to Distributed Algorithms*. Cambridge University Press, 2 edition, 2000.

[8] Mamoru Maekawa. A sqrt(n) algorithm for mutual exclusion in decentralized systems. *ACM Transactions on Computer Systems*, 3(2):145–159, 1985.

[9] Ichiro Suzuki and Tadao Kasami. A distributed mutual exclusion algorithm. *ACM Transactions on Computer Systems*, 3(4):344–349, 1985.

[10] Kerry Raymond. A tree-based algorithm for distributed mutual exclusion. *ACM Transactions on Computer Systems*, 7(1), 1989.

[11] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2), 1985.

[12] Tushar D. Chandra and Sam Toueg. Unreliable failure detectors for reliable distributed systems. *Journal of the ACM*, 43(2), 1996.

[13] Thomas Naps, Guido Rößling, Vicki Almstrum, Wanda Dann, Rudolf Fleischer, Christopher Hundhausen, Ari Korhonen, Lauri Malmi, Myles McNally, Susan Rodger, and J. Á. Velázquez-Iturbide. Exploring the role of visualization and engagement in computer science education. In *Working Group Reports from ITiCSE*, 2002.

[14] Christopher D. Hundhausen, Sarah A. Douglas, and John T. Stasko. A meta-study of algorithm visualization effectiveness. *Journal of Visual Languages and Computing*, 2002.

[15] Clifford A. Shaffer, Matthew L. Cooper, Aaron J. Alon, Monika Akbar, Michael Stewart, Sean Ponce, and Stephen H. Edwards. Algorithm visualization: The state of the field. *ACM Transactions on Computing Education*, 2010.

[16] Juha Sorva, Ville Karavirta, and Lauri Malmi. A review of generic program visualization systems for introductory programming education. *ACM Transactions on Computing Education*, 2013.

[17] Barbara Tversky, Julie Bauer Morrison, and Mireille Betrancourt. Animation: Can it facilitate? *International Journal of Human-Computer Studies*, 2002.

[18] John Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12 (2):257–285, 1988.

[19] Richard E. Mayer. *Multimedia Learning*. Cambridge University Press, 2 edition, 2009.

[20] Paul A. Kirschner, John Sweller, and Richard E. Clark. Why minimal guidance during instruction does not work: An analysis of the failure of constructivist, discovery, problem-based, experiential, and inquiry-based teaching. *Educational Psychologist*, 41(2):75–86, 2006.

[21] Jakob Nielsen. *Usability Engineering*. Morgan Kaufmann, 1994.

[22] International Organization for Standardization. Iso 9241-11: Ergonomics of human-system interaction — part 11: Usability: Definitions and concepts. International Standard, 2018.

[23] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10), 2013.

[24] Greg Wilson, D. A. Aruliah, C. T. Brown, Neil P. C. Hong, Matt Davis, Richard T. Guy, Steven H. D. Haddock, Katy D. Huff, Ian M. Mitchell, Mark D. Plumbley, Ben Waugh, Ethan P. White, and Paul Wilson. Best practices for scientific computing. *PLoS Biology*, 12(1), 2014.

[25] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, et al. The fair guiding principles for scientific data management and stewardship. *Scientific Data*, 3, 2016.

[26] IEEE. Ieee std 830-1998: Ieee recommended practice for software requirements specifications. IEEE Standard, 1998.

[27] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[28] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2 edition, 2014.

[29] W3C. Web content accessibility guidelines (wcag) 2.2. W3C Recommendation, 2023.

[30] W3C. Wai-aria 1.2. W3C Recommendation, 2021.

[31] John Brooke. Sus: A “quick and dirty” usability scale. In Patrick W. Jordan, Bruce Thomas, Bernard Weerdmeester, and Ian McClelland, editors, *Usability Evaluation in Industry*. Taylor & Francis, 1996.

Chapter A

User Manual

A.1 Running the Tool Locally

From the repository root, start a local server:

```
python -m http.server 5500
```

Open:

```
http://localhost:5500/
```

A.2 Basic Usage

- Select an algorithm (Token Ring or RA) and choose the number of processes.
- Use **Request CS** to issue a request.
- Use **Step** to progress deterministically.
- Use **Run/Pause** for automated playback.
- Observe the process table, message queue table, trace, safety label, and preview canvas.

A.3 Fault Injection

- Token Ring: use **Drop token**, **Regenerate token**, **Crash**, and **Recover**.
- RA: use **Drop next in-flight msg** and **Arm drop-next-send**.

A.4 Scripted Demos

Use the scripted demo controls to load deterministic scenarios such as crash/recover and RA tie-break demonstrations.

A.5 Evidence Export

Use the export controls to generate:

- state JSON,
- trace TXT,
- preview PNG.

These artefacts support reproducible evidence collection.

Chapter B

Test Cases Summary

This appendix summarises the minimum manual test set executed for the freeze baseline.

B.1 Token Ring

- **TR-01 Basic mutual exclusion:** issue requests from multiple processes and verify that only one process enters the critical section at a time.
- **TR-02 Token loss and regeneration:** drop the token, observe loss of progress, regenerate the token, and verify recovery.
- **TR-03 Crash and recover:** crash a process, observe state and trace changes, then recover the process.

B.2 Ricart–Agrawala

- **RA-01 Basic request/entry/release:** request the critical section and step through REQUEST/REPLY delivery until entry.
- **RA-02 Tie-break:** load the scripted tie-break scenario and verify deterministic ordering under equal timestamps.
- **RA-03 Drop next in-flight message:** drop a queued message and observe the stalled state with explicit missing-REPLY explanation.
- **RA-04 Drop-next-send:** arm the send-side drop and observe the resulting stalled state.

For each case, an evidence triplet (state JSON, trace TXT, preview PNG) is exported and indexed separately.